## Problem 3: Royal Archive Access Verification using Resolution Refutation

Group members (name, roll number):

1. Siddharth (B24EC017)
2. Parveen (B24EC013)
3. Rakesh (B24EC023)
4. Jnaneswar (B24ME027)

Files: royal_archive_access.pl (run with: swipl royal_archive_access.pl). Screenshots are in the screenshots folder.

### Task 1: Knowledge Representation

**Constants:** mira, scroll_7

**Variables:** x (a person), r (a record)

**Predicates:** archivist(x), verifies(x, r), trusted(x), has_access(x, r), sealed(r), permitted_to_inspect(x, r), inspects(x, r), finds_hidden_mark(x, r), stored_in_archive(r). Permitted_to_inspect and inspects are kept as separate predicates, because being allowed to inspect a record is not the same as actually inspecting it.

1. ∀x [ archivist(x) ∧ ∃r verifies(x, r) → trusted(x) ]
2. ∀x ∀r [ trusted(x) ∧ has_access(x, r) ∧ sealed(r) → permitted_to_inspect(x, r) ]
3. ∀x ∀r [ inspects(x, r) ∧ finds_hidden_mark(x, r) → verifies(x, r) ]
4. ∀r [ stored_in_archive(r) → sealed(r) ]
5. stored_in_archive(scroll_7)
6. has_access(mira, scroll_7)
7. inspects(mira, scroll_7) ∧ finds_hidden_mark(mira, scroll_7)
8. archivist(mira)

Goal: trusted(mira)

### Task 2: Negate the Goal

9. ¬trusted(mira) is added to the knowledge base. If this leads to a contradiction, the goal must be true.

### Task 3: Convert to Clause Form

**Implication removal (A → B becomes ¬A ∨ B):** for example, rule 1 becomes ∀x [ ¬(archivist(x) ∧ ∃r verifies(x, r)) ∨ trusted(x) ]. Rules 2, 3 and 4 change the same way.

**Negation movement:** De Morgan's law pushes the ¬ inward. In rule 1, ¬∃r turns into ∀r ¬, giving ∀x [ ¬archivist(x) ∨ ∀r ¬verifies(x, r) ∨ trusted(x) ].

**Standardization:** each formula gets its own variable names: x1, r1 in rule 1; x2, r2 in rule 2; x3, r3 in rule 3; r4 in rule 4.

**Skolemization:** no existential quantifier is left in positive position (the only ∃ became ∀ in the previous step), so no Skolem function is needed. The universal quantifiers are then dropped.

**CNF clauses:**

| Clause | Literals |
|---|---|
| C1 | ¬archivist(x1) ∨ ¬verifies(x1, r1) ∨ trusted(x1) |
| C2 | ¬trusted(x2) ∨ ¬has_access(x2, r2) ∨ ¬sealed(r2) ∨ permitted_to_inspect(x2, r2) |
| C3 | ¬inspects(x3, r3) ∨ ¬finds_hidden_mark(x3, r3) ∨ verifies(x3, r3) |
| C4 | ¬stored_in_archive(r4) ∨ sealed(r4) |
| C5 | stored_in_archive(scroll_7) |
| C6 | has_access(mira, scroll_7) |
| C7a | inspects(mira, scroll_7) |
| C7b | finds_hidden_mark(mira, scroll_7) |
| C8 | archivist(mira) |
| C9 | ¬trusted(mira) |

## Task 4: Prolog Implementation

royal_archive_access.pl follows the required structure: Step 1 (FOL facts and rules), Step 2 (negated goal), Step 3 (remove_imp), Step 4 (move_neg), Step 5 (standardize), Step 6 (skolemize), Step 7 (cnf and to_clauses) and Step 8 (unification and resolution). The conversion steps are real predicates that transform the formulas; the clauses are not typed in by hand. The program prints the result of every stage.

## Task 5: Resolution Refutation

The proof starts from the negated goal and resolves it with one input clause at each step:

| Step | Resolve | Unifier | Resolvent |
|---|---|---|---|
| R1 | C9 with C1 | {x1/mira} | ¬archivist(mira) ∨ ¬verifies(mira, r1) |
| R2 | R1 with C3 | {r1/r3, x3/mira} | ¬archivist(mira) ∨ ¬inspects(mira, r3) ∨ ¬finds_hidden_mark( mira, r3) |
| R3 | R2 with C7a | {r3/scroll_7} | ¬archivist(mira) ∨ ¬finds_hidden_mark( mira, scroll_7) |
| R4 | R3 with C7b | { } | ¬archivist(mira) |
| R5 | R4 with C8 | { } | [] (empty clause) |

The chain of reasoning: Mira inspected scroll_7 and found a hidden mark, so she verified it (rule 3). She is an archivist who verified a record, so she is trusted (rule 1). Rules 2 and 4 and facts 5

and 6 are about permission to inspect, which the goal does not need, so the proof never uses C2, C4, C5 or C6.

## Task 6: Proof Verification

The program prints the empty clause [], so ¬trusted(mira) contradicts the knowledge base. This proves trusted(mira).

swipl royal_archive_access.pl

```
$ swipl royal_archive_access.pl
...
% ======================================================================
% Step 7: CNF clauses
% ======================================================================
c1    { ¬archivist(x1) ∨ ¬verifies(x1, r1) ∨ trusted(x1) }
c2    { ¬trusted(x2) ∨ ¬has_access(x2, r2) ∨ ¬sealed(r2) ∨ permitted_to_inspect(x2, r2) }
c3    { ¬inspects(x3, r3) ∨ ¬finds_hidden_mark(x3, r3) ∨ verifies(x3, r3) }
c4    { ¬stored_in_archive(r4) ∨ sealed(r4) }
c5    { stored_in_archive(scroll_7) }
c6    { has_access(mira, scroll_7) }
c7a   { inspects(mira, scroll_7) }
c7b   { finds_hidden_mark(mira, scroll_7) }
c8    { archivist(mira) }
c9    { ¬trusted(mira) }

% ======================================================================
% Step 8: Resolution refutation
% ======================================================================
Start with the negated goal  C9: { ¬trusted(mira) }

R1 = resolve(c9, c1) on ¬trusted(mira)
     unifier θ = { x1/mira }
     resolvent: { ¬archivist(mira) ∨ ¬verifies(mira, r1) }
R2 = resolve(R1, c3) on ¬verifies(mira, r1)
     unifier θ = { r1/r3, x3/mira }
     resolvent: { ¬archivist(mira) ∨ ¬inspects(mira, r3) ∨ ¬finds_hidden_mark(mira, r3) }
R3 = resolve(R2, c7a) on ¬inspects(mira, r3)
     unifier θ = { r3/scroll_7 }
     resolvent: { ¬archivist(mira) ∨ ¬finds_hidden_mark(mira, scroll_7) }
R4 = resolve(R3, c7b) on ¬finds_hidden_mark(mira, scroll_7)
     unifier θ = { }
     resolvent: { ¬archivist(mira) }
R5 = resolve(R4, c8) on ¬archivist(mira)
     unifier θ = { }
     resolvent: []   (empty clause)

The empty clause [] was derived.
So the negated goal contradicts the knowledge base, which proves: trusted(mira)

Input clauses used: [c1,c3,c7a,c7b,c8]
Clauses not needed: [c2,c4,c5,c6]
```